Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Do Prompt-Injection Detectors Look at the Injection? A Localization-Faithful Explainability Audit of Agent Guardrails

YULLIWAS AMEUR[1,2] and SAMIA BOUZEFRANE[2]
[1]Efrei Research Lab, Efrei Paris, Université Paris-Panthéon-Assas, 94800 Villejuif, France
[2]CEDRIC Laboratory, Conservatoire National des Arts et Métiers (Cnam), 75003 Paris, France

Corresponding author: Yulliwas Ameur (e-mail: yulliwas.ameur@efrei.fr).

This work did not receive any specific funding.

ABSTRACT Classifier-based guardrails are placed in front of large language model agents to flag prompt injections hidden in tool outputs and retrieved documents, and explainable artificial intelligence is increasingly proposed to justify their verdicts. Recent work localizes injected text inside contaminated inputs or agent contexts, but does not check whether the explanations a guardrail gives for its own verdict point at the injection. This paper audits that question. We build PI-Loc, a span-annotated benchmark of 820 documents in which public injection payloads are spliced at known character offsets into benign encyclopedic passages and e-mails, together with hard negatives and clean carriers. We audit three widely used open detectors with four explainers (occlusion, KernelSHAP, attention, and gradient times input) along two axes, localization against the ground-truth span and faithfulness to the verdict, with bootstrap confidence intervals and paired tests. Detection degrades sharply once the injection is embedded: at a 5% false-positive rate the best detector recalls 61% of embedded injections and the other two recall 26% and 22%. On the discriminating detector, attention gives the best span overlap while occlusion best pinpoints the injection (its top word lies in the span 98.0% of the time, against 42.0% for attention), and both are faithful. On saturated detectors, above-chance localization is an artifact. An explanation-guided sanitization lowers the mean injection probability from 0.981 to 0.023 while deleting 12.1% of benign words. We recommend reporting embedded recall, localization and faithfulness next to accuracy, and we release the benchmark and the audit code.

INDEX TERMS Explainable artificial intelligence, faithfulness, guardrails, large language model agents, prompt injection.

I. INTRODUCTION

LARGE language model (LLM) agents act on content they did not write. A triage agent in a security operations centre reads alerts and logs; a mail assistant reads incoming messages; a retrieval-augmented assistant reads fetched web pages. Any of these inputs can carry an indirect prompt injection: instructions, planted by a third party, that the model may follow as if they came from its principal [1], [2]. Because the injected text arrives through a tool, the user never sees it, and the agent has no native notion of which span of its context is trusted.

The mitigation deployed today is a classifier-based guardrail: a text classifier placed before the agent that labels an input injection or safe [3], [4]. These detectors are cheap, model-agnostic, and report high ROC-AUC on public benchmarks of standalone prompts. As they move into security-critical pipelines, a second requirement appears: their verdicts must be explainable, so an analyst can see why a document was blocked. This is the motivation of explainable AI (XAI) for trustworthy cybersecurity.

We argue that two checks are missing before such a guardrail can be trusted, and that both require the injection to be evaluated inside a realistic document, with its ground-truth location known.

The first check is operational: a benchmark of standalone injection prompts measures whether a model can tell an injection-shaped sentence from an ordinary one, not whether it can find an injection hidden in a page of benign text. The agentic threat is the latter. We find the gap is large.

The second check is explanatory: an explanation that

highlights the trigger phrase "ignore previous instructions" is useful; one that highlights an incidental word correlated with the positive class is a liability, because it gives the analyst false confidence and tells an attacker what to edit. Measuring this needs the ground-truth span.

Span-level ground truth for injections is not new in itself. PromptLocate [5] recovers the injected instruction and data inside a contaminated input, and AttnLocate [6] marks the context spans that drive an agent's tool call. Both treat localization as the task to be solved. We use ground-truth spans for a different purpose: as a reference against which to audit the explanations of the guardrail classifiers that are deployed today, and to test whether those explanations are faithful to the verdict.

Contributions

1) **PI-LOC, a span-annotated benchmark for auditing guardrails.** We splice injection payloads, taken verbatim from public corpora [7], [8], at known character offsets into benign carriers (SQuAD passages [9] and AESLC e-mails [10]). This yields 520 indirect-injection inputs (all 520 payload strings pairwise distinct) with exact span labels, plus 180 hard negatives that contain trigger words but no instruction, and 120 clean carriers (Section III).
2) **The embedded-setting recall gap.** We show that popular guardrails either miss about half of embedded injections or flag every document, and that a calibrated threshold does not rescue the latter (Section V).
3) **A two-axis explanation audit with statistics.** We evaluate four explainers on a matched item set along localization (AUPRC, Precision@$|span|$, pointing game) and faithfulness (comprehensiveness, retained probability), with bootstrap CIs and paired Wilcoxon tests, and show that these are distinct axes on which different methods win (Section V).
4) **A measured sanitization experiment.** We remove the top-ranked words and re-query the detector, reporting the resulting injection-probability drop, the fraction of the injected span removed, and the benign collateral, and we tie the same signal to the editable shortcut an attacker exploits (Section VI).

The benchmark builder, the audit code, the raw per-item results, and the scripts that produce every number and figure of this paper are released (see the Data and Code Availability section).

Section II positions this audit against the localization methods above and against a concurrent study of guardrail saliency, Decoding Guardrails [11].

II. BACKGROUND AND RELATED WORK

Prompt injection against LLM agents

Greshake et al. [1] introduced indirect prompt injection, where the adversarial instruction is delivered through retrieved or tool-returned content. The OWASP Top 10 for LLM applications lists prompt injection as the primary risk [2]. Benchmarks such as OpenPromptInjection [12] and AgentDojo [13] evaluate attacks and defenses end to end. Labelled corpora released by deepset [7] and safe-guard [8] provide injection and benign prompts; we reuse their positive examples as payloads but change the task: the detector must find an injection inside a longer benign document.

Classifier guardrails

Lightweight DeBERTa- and DistilBERT-based detectors [3], [7], [14] and Meta's Prompt Guard family [4] are widely deployed. Their published evaluation is almost exclusively accuracy-oriented (ROC-AUC, F1 on held-out prompts), in the standalone setting. Detection methods with stronger guarantees, such as DataSentinel [15], are also evaluated on detection accuracy rather than on the evidence behind each verdict.

Localizing injected instructions

PromptLocate [5] localizes the injected instruction and the injected data inside a contaminated input, for forensic analysis and data recovery. It segments the input, uses a prompt-injection detector (a tailored DataSentinel [15]) as an oracle in a group-based binary search to find instruction-contaminated segments, and evaluates span recovery on OpenPromptInjection and on tool outputs from AgentDojo. AttnLocate [6] works inside the agent: it aggregates the agent LLM's multi-head, multi-layer attention into features, trains a detector that marks the context spans driving a tool call, and judges the call by the trust level of those spans. These works show that span-level ground truth for injections is attainable and useful. Neither asks whether the explanation a guardrail classifier gives for its own verdict points at the injection, nor whether that explanation is faithful; this is the question we audit. The two lines are connected. PromptLocate's oracle search relies on how the detector's verdict changes when segments are kept or removed, a black-box ablation close in spirit to occlusion. Our results (Section V) indicate that such ablation evidence is informative on a discriminating detector but uninformative on a saturated one, where removing words does not move the verdict. Context-traceback methods for long-context LLMs [16], [17] attribute an output to source passages; like AttnLocate, they explain the model after the injection has reached it, not the guardrail that is supposed to stop it.

XAI and its evaluation in security

Post-hoc attribution methods, namely occlusion [18], LIME [19], SHAP [20], gradient-based saliency [21], and attention-based explanations [22], are routinely applied to security classifiers. Warnecke et al. [23] showed that these methods must be judged by faithfulness (do highlighted features drive the decision?) rather than by eye; the comprehensiveness metric we use was standardized in ERASER [24]. Faithfulness does not imply localization: an explanation can be faithful to a shortcut. We measure both, against ground-truth spans.

Explanation-guided evasion

That explanations lower the cost of evasion is known for malware [23] and, concurrently with our work, for one prompt-injection guardrail [11]. Outeda et al. [11] study a single guardrail (Meta Prompt Guard 2, which is not among our three detectors) with gradient and SHAP saliency, and use saliency to evade it. We treat the same importance signal as a measurable quantity with a defensive use (sanitization, which we run) and an offensive use (shortcut editing), and tie both to how well the explanation localizes the ground-truth injection and to whether the detector is faithful at all.

Positioning

Table 1 summarizes the differences with the three closest works. Their results and ours are complementary: localization methods show that injected spans can be recovered, Decoding Guardrails shows that one guardrail relies on diffuse token evidence, and we show that popular detectors under-recall or over-flag when the injection is embedded, and that the quality of their explanations depends on both the explainer and the detector.

III. THE PI-LOC BENCHMARK

Design

PI-Loc models the indirect setting: the detector sees a document (a tool output), most of which is benign, with an injected instruction somewhere inside. Each item is built by (1) sampling a benign carrier, (2) choosing a sentence boundary uniformly at random, and (3) inserting a string there, recording its character span $[s, e)$.

Carriers

Two carrier families test distribution sensitivity: encyclopedic passages from the SQuAD v1.1 training split [9] ("wiki"), and e-mail bodies from the AESLC training split [10] ("email"). Carriers have 70–180 words and contain no word of a fixed trigger lexicon (e.g., ignore, previous, instructions, rules, prompt); items alternate between the two families, and no carrier is used twice.

Payloads

To keep every attack string a citable public artifact rather than a newly authored one, payloads are drawn verbatim from the positive (injection) rows of the deepset [7] and safeguard [8] corpora, keeping English strings of 5–55 words (language identified automatically). The two corpora yield 2,665 distinct positive strings, from which the 520 injections draw 520 pairwise-distinct payloads.

Hard negatives and clean carriers

A hard negative is a carrier with an inserted benign sentence of 8–40 words that contains trigger lexicon. These sentences are encyclopedic statements taken from SQuAD passages that contain a trigger word (for example, a sentence about "the previous three seasons"); such passages are never used as carriers. Hard negatives separate "detecting the word previous" from "detecting an instruction". Clean carriers are left unchanged.

Classes and release

PI-Loc has 820 items: 520 injections (label 1, with spans), 180 hard negatives and 120 clean carriers (label 0). The builder is seeded and pins every dataset to an exact revision; it reproduces the benchmark file byte for byte, which the released tests check with a SHA-256 digest.

IV. AUDIT METHODOLOGY

Detectors

We audit three open, widely downloaded classifier guardrails, pinned to exact model revisions: protectai/deberta-v3-base-prompt-injection-v2 [3], deepset/deberta-v3-base-injection [7], and fmops/distilbert-prompt-injection [14]. Each returns $P(\text{injection} \mid x)$; inputs are truncated at 512 tokens, which no PI-Loc document reaches.

Explainers

Every explainer produces a word-level importance vector over the whitespace-separated words of the document. Two are black-box. Occlusion: the importance of word w_i is the drop in $P(\text{injection})$ when w_i is removed [18]. KernelSHAP: random word coalitions, weighted with the SHAP kernel and fitted with weighted ridge regression [20] (96 coalitions on the anchor detector, 48 on the others). Two are white-box. Attention: mean over heads of the last-layer attention from the first position to each token, max-pooled to words [22]. Gradient×input: the absolute value of the dot product between the input embedding and the gradient of $P(\text{injection})$ with respect to it, max-pooled to words [21].

Matched item set

Localization is defined only on true positives. We therefore fix one evaluation set, a seeded sample of 50 of the 245 injections caught by the strongest detector (`protectai`), and score all three detectors on the same items, so that cross-detector cells are comparable (Appendix).

Localization metrics

For each item we compare the word-importance vector to the binary in-span word mask: AUPRC (ranking words by importance), Precision@$|span|$ (fraction of the top-$|span|$ words in-span), and the pointing game (is the single most important word in-span?) [25]. The chance AUPRC is the mean fraction of in-span words over the evaluated items, 0.129.

Faithfulness metrics

Comprehensiveness is the drop in $P(\text{injection})$ when the top-20% words are removed (ERASER [24]; higher is better). We also report the retained probability when only the top-20% words are kept (a sufficiency proxy; higher means the top words alone suffice).

TABLE 1. This work compared with the closest related studies.

Dimension	*Decoding Guardrails* [11]	PromptLocate [5]	AttnLocate [6]	This work
What is explained or located	guardrail verdict	injected text in an input	spans driving a tool call	guardrail verdict
Ground-truth injection span	no	yes	yes	yes (PI-LOC)
Setting	standalone prompts	contaminated inputs, tool outputs	agent context	injections embedded in benign documents
Guardrail classifiers audited	1 (Prompt Guard 2)	detector used as an oracle	—	3 (none shared)
Explainers compared	2 (gradient, SHAP)	attribution baselines	attribution baselines	4 (occlusion, SHAP, attention, gradient)
Faithfulness of the explanation	not measured	not studied	not studied	comprehensiveness, retained probability
Use of the signal	evasion (run)	forensics, data recovery	tool-call adjudication	sanitization (run), evasion (discussed)

Statistics

We report 95% bootstrap confidence intervals (5000 resamples) for AUPRC and comprehensiveness, Wilson intervals for the pointing game, and paired Wilcoxon signed-rank tests between explainers. Experiments run on CPU with a fixed seed; re-running them in a fresh process reproduces every per-item metric exactly.

V. RESULTS

A. DETECTION COLLAPSES IN THE EMBEDDED SETTING

Table 2 and Fig. 1 report detection on PI-Loc. The three detectors fail in two different ways. `protectai-deberta-v2` ranks documents well (ROC-AUC 0.893) and never false-alarms on hard negatives or clean carriers at its default 0.5 threshold, yet it recalls only 47.1% of the embedded injections: a classifier tuned on standalone prompts misses more than half of the same payloads once they sit inside a benign page. `deepset-deberta` and `fmops-distilbert` do the opposite: at the default threshold they label almost every document an injection, including all hard negatives. Their scores still carry some ranking information (ROC-AUC 0.766 and 0.749), so we also report recall at a calibrated operating point, TPR at FPR$\leq$0.05, with the threshold set on the negatives (last column of Table 2). It is 0.610 for protectai but only 0.258 and 0.223 for deepset and fmops. Even with a usable false-alarm budget, these two detectors therefore miss about three embedded injections out of four; the failure is not merely a wrong threshold. Only one of three popular guardrails is a usable discriminator in the agentic setting, and even it misses 39% of embedded injections at that budget.

TABLE 2. Detection on PI-LOC. Columns 2–5 use the default 0.5 threshold (TPR on embedded injections; FPR on hard negatives and clean carriers). The last column is recall at a threshold calibrated so that at most 5% of negatives are flagged.

Detector	ROC-AUC	TPR_{INJ}	FPR_{HN}	$\text{FPR}_{\text{CLEAN}}$	$\text{TPR@FPR}_{.05}$
protectai-deberta-v2	0.893	0.471	0.000	0.000	**0.610**
deepset-deberta	0.766	1.000	1.000	1.000	0.258
fmops-distilbert	0.749	1.000	1.000	0.967	0.223

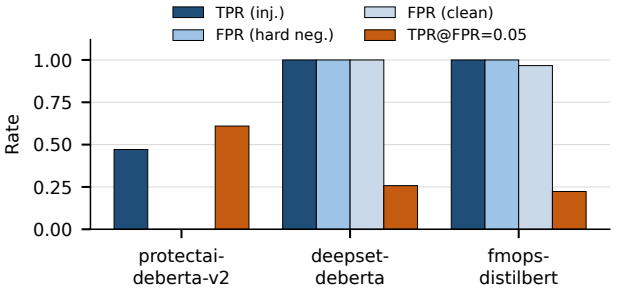


FIGURE 1. Detection rates on PI-LOC (values of Table 2). High ROC-AUC hides the embedded-setting behavior: low recall (protectai) or indiscriminate flagging (deepset, fmops) at the default threshold.

B. LOCALIZATION: ABOVE CHANCE, BUT METHOD-DEPENDENT

Table 3 and Fig. 2 report localization for the one detector that actually discriminates, `protectai-deberta-v2`; localization on a detector that flags everything is not meaningful (Section VII and Appendix). Every explainer beats the chance AUPRC of 0.129, so the detector does place weight inside the injected span. But the methods disagree on which notion of localization they satisfy. Attention has the best span overlap (AUPRC 0.636, CI $[0.575, 0.698]$; Precision@$|span|$ 0.634): its mass is spread across the span. Occlusion has lower overlap (AUPRC 0.444, CI $[0.412, 0.475]$) but by far the best pinpointing: its single most important word lands inside the injection 98.0% of the time (Wilson CI $[0.895, 0.996]$), against 42.0% for attention. Both contrasts are significant (paired Wilcoxon, AUPRC $p < 10^{-5}$, pointing $p < 10^{-5}$). Which method to prefer therefore depends on the downstream use: highlighting a region for an analyst (favor attention) or selecting the exact tokens to remove (favor occlusion). KernelSHAP and gradient×input trail on both notions; with our coalition budget, KernelSHAP is barely above chance.

TABLE 3. Localization against ground-truth spans on `protectai-deberta-v2` ($n = 50$ matched true-positive injections; 95% CI). Chance AUPRC $= 0.129$.

Explainer	AUPRC (CI)	P@$\|span\|$	Pointing game (CI)
Occlusion	0.444 [0.412,0.475]	0.400	**0.980** [0.895,0.996]
KernelSHAP	0.185 [0.163,0.206]	0.159	0.220 [0.128,0.352]
Grad.×input	0.389 [0.351,0.430]	0.336	0.620 [0.482,0.741]
Attention	**0.636** [0.575,0.698]	**0.634**	0.420 [0.294,0.558]

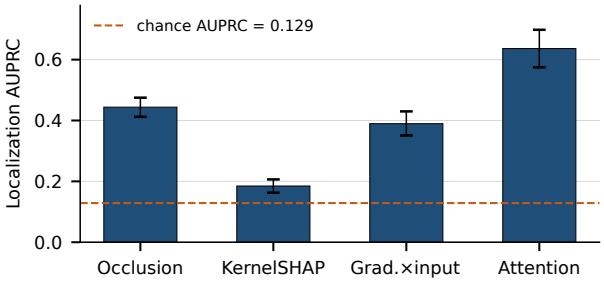


FIGURE 2. Localization AUPRC by explainer on the discriminating detector, with 95% bootstrap CIs (values of Table 3). All beat chance (dashed); attention leads on span overlap.

TABLE 4. Faithfulness on `protectai-deberta-v2` ($n = 50$; comprehensiveness with 95% CI). On the two over-flagging detectors comprehensiveness is ≈ 0 (probability saturated); see the Appendix.

Explainer	Comprehensiveness↑ (CI)	Retained prob. (top-20%)↑
Occlusion	0.981 [0.965,0.992]	0.959
KernelSHAP	0.431 [0.302,0.563]	0.380
Grad.×input	0.665 [0.539,0.782]	0.664
Attention	0.958 [0.911,0.987]	1.000

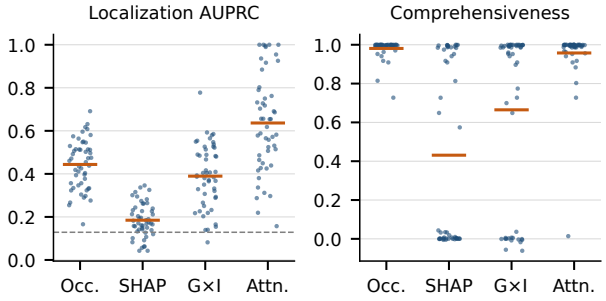


FIGURE 3. Per-item localization AUPRC (left) and comprehensiveness (right) on `protectai-deberta-v2` ($n = 50$; one point per injection, bar = mean, dashed = chance).

C. FAITHFUL, LOCALIZED, AND WHY BOTH ARE NEEDED

Table 4 shows why an above-chance localization score cannot be read on its own. On the discriminating detector, occlusion and attention are both highly faithful (comprehensiveness 0.981 and 0.958: removing their top words drives the injection probability to near zero). Occlusion is slightly more faithful ($p = 5.6 \times 10^{-5}$), but attention localizes the span better, so a defender who wants faithful and well-covered evidence should combine attention's overlap with occlusion's pinpointing. On `deepset-deberta` and `fmops-distilbert`, by contrast, comprehensiveness is ≈ 0 for every explainer, even though several of their explanations still localize above chance (Appendix): because the probability is saturated near 1, no small set of words drives the verdict, and the "localization" is an artifact of the attribution method, not evidence that the detector reasons about the span. Localization and faithfulness are therefore distinct, and both are required. Fig. 3 shows the per-item spread behind these means.

VI. THE DUAL USE OF WORD IMPORTANCE

Defensive use: explanation-guided sanitization (measured)

We run sanitization end to end on `protectai-deberta-v2`: for each caught injection we rank words by an explainer, delete the top-k%, and re-query the detector on the cleaned document. Table 5 reports, for $k \in \{10, 20\}$, the resulting injection probability, its drop, the fraction of the injected span removed, and the benign collateral. Removing the top-20% attention-ranked words lowers the mean injection probability from 0.981 to 0.023, excising 80.6% of the injected span while deleting 12.1% of benign words; occlusion at 20% reaches a probability of 0.000 at 16.8% collateral. At k=10% the collateral falls to 6.3% (occlusion) and 4.9% (attention); occlusion still drives the probability to 0.020, while attention leaves it at 0.119. Sanitization is thus effective but not free, and its quality presupposes the faithful regime of Section V: on the saturated detectors, where comprehensiveness is ≈ 0, no word removal moves the verdict, matching the fact that those detectors are not usable gates in the first place.

Offensive use: the shortcut is editable

The mirror image is the risk that Decoding Guardrails [11] raises: an attacker reading the same importance vector knows which tokens carry the verdict and can rewrite them while preserving intent. Our contribution is to connect the two: a detector that localizes and is faithful exposes a specific editable target, whereas the low recall of `protectai` in the embedded setting (Table 2) means that many injections never trigger it at all. Transparency and robustness trade off, and the trade-off is measurable on PI-Loc.

Remark

A guardrail's ROC-AUC on standalone prompts predicts neither its recall on embedded injections nor whether its explanations point at them. Both must be measured before deployment behind an agent.

VII. DISCUSSION AND RECOMMENDATIONS

Report embedded recall and localization next to ROC-AUC

For any guardrail proposed to protect an agent, we recommend reporting (i) recall on injections embedded in benign

TABLE 5. Explanation-guided sanitization on `protectai-deberta-v2` ($n = 50$): delete the top-$k\%$ words by importance and re-query. Mean $P(\text{INJ})$ before removal is 0.981.

Explainer	k	$P(\text{INJ})$ after	Drop	Span removed	Collateral
Occlusion	10%	0.020	0.961	0.396	0.063
Occlusion	20%	0.000	0.981	0.454	0.168
Attention	10%	0.119	0.862	0.557	0.049
Attention	20%	0.023	0.958	0.806	0.121

carriers and (ii) a localization score on a span-annotated benchmark such as PI-Loc, alongside detection accuracy. A detector that cannot be shown to look at the injection should not be trusted to explain itself.

Choose the explainer for the use

Occlusion pinpoints the injection and is the most faithful explainer here (favor it for token-level audit and for low-collateral sanitization at small k); attention gives the best region overlap and removes the largest share of the span at k=20% (favor it for analyst highlighting), but its single peak is often off-span and should not be used to select one token. KernelSHAP and gradient×input were weaker on both axes.

Faithfulness gates localization

An above-chance localization number on a detector whose probability is saturated is an artifact. Localization should be read only together with a faithfulness check. This also applies to localization methods that query a detector as an oracle [5]: their evidence is only as good as the detector's sensitivity to the removed text.

Limitations

The audit uses three open encoder classifiers, four explainers, and $n = 50$ matched items from a single benchmark draw; we mitigate the sample size with bootstrap and Wilson intervals and paired tests, but larger n, several benchmark draws, newer guardrails such as Prompt Guard 2, and closed or generative judge-style guardrails are future work. KernelSHAP is sensitive to its coalition budget, and a larger budget may improve it. PI-Loc payloads are reused from two English corpora and spliced at sentence boundaries; multilingual, obfuscated, and natural (non-spliced) injections remain open, as does calibrating a detector's operating threshold on embedded rather than standalone data. We do not compare against dedicated localization methods [5], [6], which solve a different task; using them as additional explainers of a guardrail verdict is a natural extension.

VIII. CONCLUSION

We asked whether prompt-injection detectors look at the injection, and built the span-annotated benchmark needed to answer it. Across three open guardrails and four explainers we found that headline ROC-AUC hides two failures: popular detectors either miss about half of embedded injections or flag almost every document, and explanation quality splits into two distinct, separately significant axes. Attention best covers the injected span while occlusion best pinpoints it, both faithful, whereas an above-chance localization score is meaningless on a detector whose decisions no small word set drives. The same importance signal that enables measured document sanitization on a sound detector also exposes the shortcut an attacker can edit. Trustworthy guardrails for LLM agents should therefore be evaluated on where they look and on whether they still fire once the injection is embedded, not only on how often they are right on standalone prompts.

.

APPENDIX A PER-DETECTOR LOCALIZATION AND FAITHFULNESS (MATCHED ITEMS)

Table 6 gives the full breakdown over all three detectors and four explainers, all scored on the same 50 injections (a seeded sample of those caught by `protectai`). On the two over-flagging detectors (deepset, fmops), comprehensiveness is ≈ 0: no small word set drives the (saturated) verdict, so their above-chance localization AUPRC must not be read as evidence of span-aware reasoning. This is why the main-text localization and faithfulness tables are reported on the discriminating detector; the comparison here is what licenses that choice.

TABLE 6. Full audit on the matched item set ($n = 50$). Loc. = localization AUPRC (chance 0.129); PG = pointing game; Comp. = comprehensiveness.

Detector	Explainer	Loc.	PG	Comp.
protectai-deberta-v2	Occlusion	0.444	0.980	0.981
	KernelSHAP	0.185	0.220	0.431
	Grad.×input	0.389	0.620	0.665
	Attention	0.636	0.420	0.958
deepset-deberta	Occlusion	0.394	0.540	0.002
	KernelSHAP	0.177	0.200	0.000
	Grad.×input	0.228	0.400	0.000
	Attention	0.351	0.240	0.000
fmops-distilbert	Occlusion	0.338	0.580	0.005
	KernelSHAP	0.174	0.120	0.000
	Grad.×input	0.414	0.460	0.007
	Attention	0.448	0.540	0.016

DATA AND CODE AVAILABILITY

The PI-Loc generator, the full benchmark file, the audit harness, the statistics scripts, and the aggregate results are available at https://github.com/yulliwasameur/pi-loc and archived at https://doi.org/10.5281/zenodo.24085151. All attack payloads are reused verbatim from the cited public benchmarks [7], [8]; no new attack strings were authored.

ACKNOWLEDGMENT

An AI assistant (Claude, Anthropic) was used extensively in this work. At the first author's request and under his direction, it wrote the code that builds PI-Loc and runs the audit, executed the experiments, and drafted the manuscript. Every number in the article is generated from the released

result files by a script, not typed by hand. Y. Ameur initiated the study and reviewed the design, code, results, and text; S. Bouzefrane critically reviewed the study and the manuscript. Both authors approved the final version and take full responsibility for its content.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, pp. 79–90.

[2] OWASP Foundation, "OWASP top 10 for large language model applications," https://genai.owasp.org/llm-top-10/, 2025.

[3] Protect AI, "Fine-tuned DeBERTa-v3-base for prompt injection detection (v2)," https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2, 2024.

[4] Meta AI, "Prompt guard," https://huggingface.co/meta-llama/Prompt-Guard-86M, 2024.

[5] Y. Jia, Y. Liu, Z. Shao, J. Jia, and N. Z. Gong, "PromptLocate: Localizing prompt injection attacks," in Proc. IEEE Symp. Security and Privacy (S&P), 2026, arXiv:2510.12252.

[6] Y. Gao, Y. Zhang, Y. Yao, H. Du, P. Luo, R. Li, and Z. Wang, "What guides the agent? Adjudicating unauthorized behavior via localizing behavior-guiding instructions," 2026, arXiv:2608.24022.

[7] deepset, "Prompt injections: A labelled dataset and DeBERTa injection classifier," https://huggingface.co/datasets/deepset/prompt-injections, 2023.

[8] xTRam1, "Safe-guard prompt injection dataset," https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection, 2024.

[9] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2016, pp. 2383–2392.

[10] R. Zhang and J. Tetreault, "This email could save your life: Introducing the task of email subject line generation," in Proc. 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 446–456.

[11] F. Outeda, G. Betarte, J. D. Campo, and F. Cravero, "Decoding guardrails: XAI-guided perturbation analysis of prompt injection detection," 2026, arXiv:2609.24801.

[12] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in Proc. 33rd USENIX Security Symposium, 2024.

[13] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2024.

[14] fmops, "DistilBERT prompt injection classifier," https://huggingface.co/fmops/distilbert-prompt-injection, 2023.

[15] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in Proc. IEEE Symp. Security and Privacy (S&P), 2025.

[16] Y. Wang, W. Zou, R. Geng, and J. Jia, "TracLLM: A generic framework for attributing long context LLMs," 2025, arXiv:2506.04202.

[17] Y. Wang, R. Geng, Y. Chen, and J. Jia, "AttnTrace: Contextual attribution of prompt injection and knowledge corruption," 2025, arXiv:2508.03793.

[18] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 818–833.

[19] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[21] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in Proc. 34th Int. Conf. Machine Learning (ICML), 2017, pp. 3319–3328.

[22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

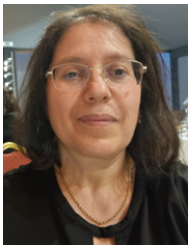
[23] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in Proc. IEEE European Symp. Security and Privacy (EuroS&P), 2020, pp. 158–174.

[24] J. DeYoung, S. Jain, N. F. Rajani, E. Lehman, C. Xiong, R. Socher, and B. C. Wallace, "ERASER: A benchmark to evaluate rationalized NLP models," in Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4443–4458.

[25] J. Zhang, S. A. Bargal, Z. Lin, J. Brandt, X. Shen, and S. Sclaroff, "Top-down neural attention by excitation backprop," International Journal of Computer Vision, vol. 126, no. 10, pp. 1084–1102, 2018.



YULLIWAS AMEUR received the Ph.D. degree in computer science from the Conservatoire National des Arts et Métiers (Cnam), Paris, France, in 2023. He is a permanent Lecturer-Researcher in cybersecurity with the Efrei Research Lab, Efrei Paris, Université Paris-Panthéon-Assas, Villejuif, France, and an associate member of the CEDRIC Laboratory, Cnam. His research interests include the security of AI agents, prompt injection, privacy-preserving computation with homomorphic encryption, and post-quantum cryptography.



SAMIA BOUZEFRANE is a Full Professor at the Conservatoire National des Arts et Métiers (Cnam), Paris, France, where she heads the CEDRIC Laboratory and is a member of its Networks and Connected Objects team. Her research interests include cybersecurity and smart-card systems.